

TRAVAUX PRATIQUE EXAMEN - ICT308 - DUREE : 10 JOURS à PARTIR DU 27 JUILLET

I. Organisation et Répartition du Groupe de 10 Étudiants (Par Thème) Dans l'ordre alphabétique.

Pour chaque TP, le groupe de 10 étudiants doit se structurer comme une véritable équipe technique agile, divisée en **4 sous-groupes spécialisés** :

1. **Équipe Core & Métier (3 étudiants)** : Modélisation de la POO, classes abstraites, héritage, interfaces, algorithmes fondamentaux et gestion des collections.
2. **Équipe Persistance & Données (2 étudiants)** : Écriture des modules d'accès aux données (Fichiers Sérialisés ou DAO JDBC/MySQL).
3. **Équipe IHM Swing (3 étudiants)** : Conception des fenêtres, agencement des composants (**BorderLayout**, **GridLayout**), JTable et gestionnaires d'événements.
4. **Équipe Performance & Multithreading (2 étudiants)** : Traitements asynchrones (**Thread**, **Runnable**, **SwingWorker**), synchronisation des données et barres de progression dynamiques.

Projet 1 : Système de Gestion du Trafic de Fret Portuaire (Port Autonome de Douala)

- **Thématique principale** : POO de base, Polymorphisme, Collections (**ArrayList**), Gestion des fichiers Sérialisés.
- **Contexte** : Le Port Autonome de Douala souhaite automatiser le suivi des marchandises arrivant par bateau. Les conteneurs peuvent être de type "Standard" ou "Réfrigéré" (nécessitant un contrôle permanent de la température).
- **Cahier des charges** :
 - **Métier** : Créer une classe abstraite **Marchandise** et des sous-classes **ConteneurStandard** et **ConteneurRefrigerere**. Implémenter le polymorphisme sur le calcul des taxes portuaires (les réfrigérés payent un supplément énergétique).
 - **Persistance** : Sauvegarder et charger la liste globale des cargaisons dans un fichier binaire **fret.ser** via la sérialisation d'objets.
 - **IHM** : Une interface Swing affichant la liste complète dans une **JTable** avec un bouton permettant de filtrer les cargaisons en alerte (température hors norme).
 - **Multithreading** : Un thread d'arrière-plan simule la variation de température des conteneurs réfrigérés toutes les 3 secondes et met à jour l'IHM en temps réel.

Projet 2 : Plateforme de Suivi des Consommations d'Énergie Post-payées (ENEO)

- **Thématique principale** : Encapsulation stricte, Exceptions personnalisées, Tri et expressions Lambdas, IHM Swing.
- **Contexte** : Pour numériser la relève des index, une application doit permettre aux techniciens de saisir les consommations de kWh des abonnés (catégories Social, Résidentiel, Industriel) et calculer les factures avec la TVA.
- **Cahier des charges** :

- **Métier** : Modélisation de la classe **Abonné**. Créer une exception personnalisée **IndexIncoherentException** levée si le nouvel index saisi est inférieur à l'ancien index.
- **Traitement** : Utiliser les expressions Lambdas et l'API Stream pour trier les abonnés par montant de facture décroissant et filtrer les gros consommateurs.
- **Persistance** : Exportation des factures impayées au format texte structuré (**.csv**) pour relance.
- **IHM** : Formulaire de saisie ergonomique avec gestion des raccourcis clavier (**Entrée** pour valider) et boîtes de dialogue **JOptionPane** descriptives en cas d'erreur de saisie.

Projet 3 : Simulateur Automobile de file d'attente aux Péages Routiers

- **Thématique principale** : Multithreading intensif, Blocs Synchronized (**wait** / **notifyAll**), Collections (**LinkedList** comme file).
- **Contexte** : Sur l'axe Yaoundé-Douala, le ministère des transports veut simuler l'impact de la création de nouveaux guichets de péage automatique pour désengorger le trafic.
- **Cahier des charges** :
 - **Métier** : Classe **Vehicule** (Camion, Bus, Berline) et classe **Peage** possédant une file d'attente de véhicules.
 - **Multithreading** : Un thread **GenerateurTrafic** injecte des véhicules dans la file à des intervalles aléatoires (200ms à 800ms). Plusieurs threads **GuichetPeage** traitent (encaissent) les véhicules en tête de file.
 - **Synchronisation** : Si la file est vide, les threads guichets se mettent en attente (**wait()**). Dès qu'un véhicule arrive, le générateur réveille les guichets (**notifyAll()**). Le solde de la caisse centrale doit être protégé contre les accès concurrents (**synchronized**).
 - **IHM** : Visualisation graphique de la longueur de la file sous forme de **JProgressBar** dynamique.

Projet 4 : Système Bancaire Centralisé de Transfert Mobile (Mobile Money Core API)

- **Thématique principale** : JDBC avancé, Transactions SQL, Pattern DAO, Sécurité Swing.
- **Contexte** : Micro-implémentation d'un cœur de réseau financier gérant les dépôts, retraits et transferts de compte à compte avec prélèvement de commissions.
- **Cahier des charges** :
 - **Base de données** : Tables MySQL **comptes** (numero, solde, titulaire) et **transactions** (id, source, destination, montant, date).
 - **Métier & JDBC** : Implémenter le pattern **DAO**. Pour l'opération **transférer()**, utiliser une transaction SQL (**conn.setAutoCommit(false)**) afin de garantir que le débit du compte A et le crédit du compte B s'exécutent entièrement ou pas du tout (Rollback en cas d'échec).
 - **IHM** : Fenêtre de connexion sécurisée pour l'agent de guichet avec un **JPasswordField** (bouton bloqué après 3 tentatives infructueuses).

- **Multithreading** : Utilisation d'un `SwingWorker` pour charger les relevés bancaires lourds depuis MySQL sans figer l'interface visuelle.

Projet 5 : Logiciel de Planification Clinique et Gestion des Urgences Hospitalières

- **Thématique principale** : Interfaces (`Comparable`), Structures de données (`PriorityQueue` ou tri automatique), JDBC.
- **Contexte** : Gestion des admissions au sein des urgences d'un hôpital. Les patients doivent être triés non pas par ordre d'arrivée, mais selon un niveau de priorité médicale (Code Rouge, Orange, Vert).
- **Cahier des charges** :
 - **Métier** : La classe `Patient` implémente l'interface `Comparable<Patient>`. La méthode `comparerAvec()` compare la sévérité de la pathologie puis l'heure d'arrivée.
 - **Base de données** : Persistance JDBC permanente des dossiers médicaux admis et archivés.
 - **IHM** : Tableau Swing (`JTable`) mis à jour automatiquement. Un panneau de commandes permet au médecin de cliquer sur un bouton "Appeler prochain patient" pour extraire instantanément le patient le plus urgent.
 - **Multithreading** : Une horloge interne affiche le temps d'attente moyen des patients actuellement en salle d'attente.

Projet 6 : Système de Vente et d'Impression des Billets de Train (Camrail Intercity)

- **Thématique principale** : Architecture Swing complète (`CardLayout`, `BorderLayout`), Flux d'écriture de fichiers textes (`PrintWriter` / `FileWriter`).
- **Contexte** : Borne interactive permettant aux voyageurs de réserver une place de train en sélectionnant la classe (Première / Business / Économique) et émettant un reçu physique officiel.
- **Cahier des charges** :
 - **IHM** : Utilisation de `CardLayout` pour concevoir un assistant d'achat étape par étape (Écran 1 : Choix du trajet, Écran 2 : Choix du siège, Écran 3 : Paiement et impression).
 - **Persistance** : À la validation du paiement, le système génère un fichier texte structuré `ticket_XXXXXX.txt` imitant la mise en forme d'un reçu de caisse (avec entête, date, détails des taxes et code de sécurité unique).
 - **Multithreading** : L'IHM affiche une animation visuelle simulant "Impression du billet en cours..." pendant 2.5 secondes avant de réinitialiser l'application.
 - **Exceptions** : Capturer proprement les erreurs matérielles simulées (ex: fichier verrouillé ou inaccessible).

Projet 7 : Moteur d'Indexation et d'Analyse de Fichiers Logs de Serveurs Web (Cybersécurité)

- **Thématique principale** : Traitement de fichiers texte lourds (`BufferedReader`), Collections avancées (`HashMap`), Expressions Lambdas.

- **Contexte :** Les administrateurs d'un centre de données reçoivent des milliers de lignes de logs au format Apache/Nginx. L'application doit analyser ces fichiers pour recenser le nombre d'attaques par adresse IP.
- **Cahier des charges :**
 - **Persistance :** Lecture ligne par ligne d'un fichier de log textuel via `BufferedReader` pour éviter de surcharger la mémoire RAM.
 - **Métier :** Utilisation d'une collection `Map<String, Integer>` (ou `HashMap`) pour comptabiliser dynamiquement les occurrences de chaque adresse IP ou chaque type d'erreur HTTP (ex: 404, 500).
 - **Lambdas :** Filtrer et extraire le "Top 5" des adresses IP ayant généré le plus de requêtes suspectes à l'aide des Streams Java.
 - **IHM :** Affichage d'un bilan statistique visuel brut avec option de recherche textuelle dynamique dans les logs.

Projet 8 : Application de Supervision Météo et d'Alerte Agricole en Réseau

- **Thématique principale :** Simulation de capteurs concurrents, Multithreading, Interfaces d'écoute personnalisées (Pattern Observer).
- **Contexte :** Des capteurs connectés mesurent l'humidité du sol et la température dans une plantation de cacaoyers pour déclencher des systèmes d'irrigation automatiques.
- **Cahier des charges :**
 - **Multithreading :** Créer une classe `CapteurMeteo` implémentant `Runnable`. Chaque instance représente une zone de la plantation et génère des mesures météorologiques fluctuantes de manière autonome.
 - **Métier :** Créer une interface `MeteoListener` possédant la méthode `onMesureRecue(double temp, double humidite)`. L'application principale s'abonne aux capteurs pour centraliser les données.
 - **IHM :** Tableau de bord Swing présentant les indicateurs de toutes les zones en parallèle. Si l'humidité d'une zone descend sous le seuil critique de 30%, la ligne correspondante dans la table passe visuellement au rouge vif et un signal sonore/visuel d'alerte est déclenché.

Projet 9 : Système d'Enregistrement et Répartition des Notes d'Examen Universitaires (Scolarité)

- **Thématique principale :** JDBC complet (CRUD), Agencements de formulaires complexes (`GridBagLayout` ou multiples `JPanel`), Traitement de listes.
- **Contexte :** Application interne de scolarité pour l'Université de Yaoundé I, permettant la saisie des notes de CC et d'Examen des étudiants pour une UE, avec calcul automatique des moyennes pondérées, des mentions et des statuts (Admis / Rattrapage).
- **Cahier des charges :**
 - **Base de données :** Table relationnelle complète contenant les informations d'identification de l'étudiant et ses notes.
 - **Métier & JDBC :** Implémentation complète des méthodes CRUD : `insérer()`, `lireTous()`, `mettreAJour()` et `supprimer()` via un objet `EtudiantDAO` robuste utilisant des requêtes préparées (`PreparedStatement`).

- **IHM** : Interface Swing hautement soignée mariant un `GridLayout` pour les champs de saisie de notes, un `BorderLayout` structurel et un panneau de statistiques globales calculées en SQL (`COUNT`, `AVG`).

Projet 10 : Gestionnaire de Téléchargement Parallèle de Fichiers Multimédia

- **Thématique principale** : Cycle de vie des threads (`start`, `join`, états de threads), `SwingWorker` ou threads UI, Collections thread-safe.
- **Contexte** : Outil utilitaire permettant de lancer plusieurs téléchargements ou copies simultanées de fichiers volumineux avec possibilité de suivre la progression individuelle de chaque tâche.
- **Cahier des charges** :
 - **Métier** : Classe `TâcheTelechargement` implémentant `Runnable` possédant un nom, une taille totale et une progression courante.
 - **Multithreading** : L'utilisateur clique sur un bouton pour ajouter une tâche. Chaque téléchargement tourne dans un thread indépendant. Utilisation appropriée de `Thread.sleep()` pour cadencer le débit simulé et de `join()` si une tâche globale nécessite la fin des sous-tâches.
 - **IHM** : Ajout dynamique de lignes dans l'IHM Swing. Chaque ligne contient le nom du fichier et une `JProgressBar` dédiée dont la valeur est rafraîchie de façon sécurisée via `SwingUtilities.invokeLater()` ou un framework de communication asynchrone.

Projet 11 : Système de Réservation et d'Allocation de Salles de Co-working

- **Thématique principale** : Ensembles sans doublon (`HashSet`), Persistance mixte, Gestion avancée des dates et chevauchements d'horaires.
- **Contexte** : Une entreprise tech loue des salles de réunion à l'heure. L'application doit interdire de réserver une même salle pour des plages horaires en conflit.
- **Cahier des charges** :
 - **Métier** : Classe `Reservation`. Utilisation d'un `Set<Reservation>` (implémenté via `HashSet`) pour stocker les réservations de la journée. Redéfinir obligatoirement les méthodes fondamentales `equals()` et `hashCode()` héritées de la classe `Object` pour qu'un doublon soit automatiquement rejeté par la collection si les identifiants ou les plages horaires coïncident.
 - **Persistance** : Sauvegarde automatique des données lors de la fermeture de l'application dans un fichier plat indexé.
 - **IHM** : Calendrier visuel simplifié (grille horaire interactive) où le clic sur une cellule ouvre le formulaire de réservation pour la plage horaire sélectionnée.

II. Modèle de Grille d'Évaluation de l'Examen (Sur 20 Points)

Pour l'enseignant, ce barème s'applique à la démonstration finale et à l'examen du code source écrit par le groupe:

Critères d'Évaluation	Points	Indicateurs Clés Attendus
Architecture & Qualité POO	/ 4 pts	Respect de l'encapsulation (private), hiérarchies claires, bon usage des classes abstraites/interfaces.
Interface Graphique Swing	/ 4 pts	Alignement propre via les Layouts, redimensionnement stable, composants adéquats, messages d'erreurs clairs.
Mécanique Multithreading	/ 3 pts	Réactivité de l'IHM préservée, absence de verrous mortels (<i>deadlocks</i>), bonne synchronisation des ressources.
Persistance des Données	/ 3 pts	Opérations CRUD JDBC ou sérialisation exemptes d'erreurs, fermeture propre des flux/connexions SQL.
Robustesse (Exceptions)	/ 2 pts	Captures ciblées des erreurs (try-catch), aucun plantage de l'application face à de mauvaises saisies.
Soutenance & Maîtrise Individuelle	/ 4 pts	Capacité de chaque étudiant à expliquer précisément sa contribution et la logique du code du groupe.
NOTE FINALE	/ 20 pts	